



**Laboratorium
Multimedia dan Internet of Things
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember**

Laporan Akhir Praktikum Jaringan Komputer

Firewall & NAT

Bagas Aryo Dananjoyo - 5024241031

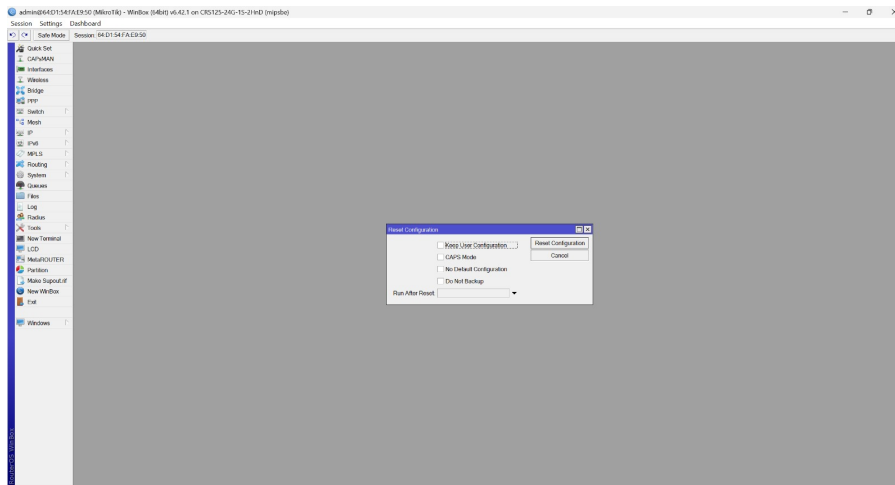
2026

1 Langkah-Langkah Percobaan

Praktikum 1: Firewall NAT Dasar

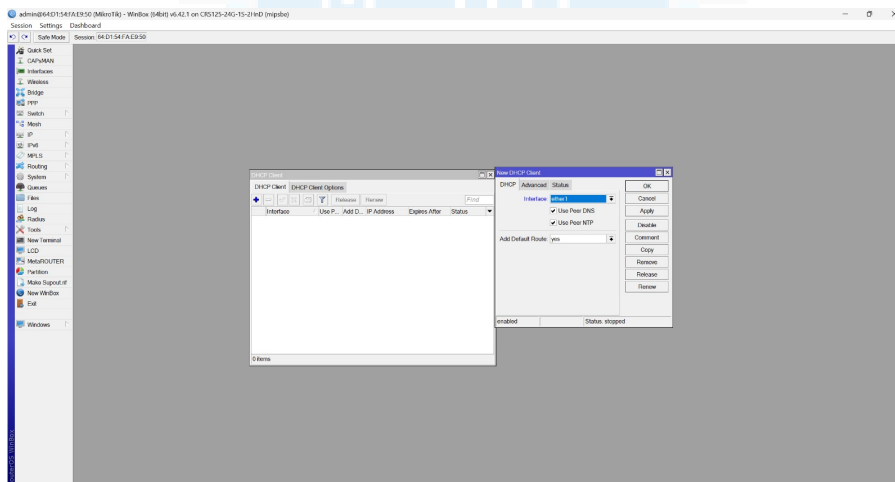
Langkah 1: Reset Konfigurasi Router

Memastikan Router A dan Router B di-reset tanpa konfigurasi bawaan (*No Default Configuration*) melalui Winbox agar tidak terjadi tumpang tindih aturan lama.



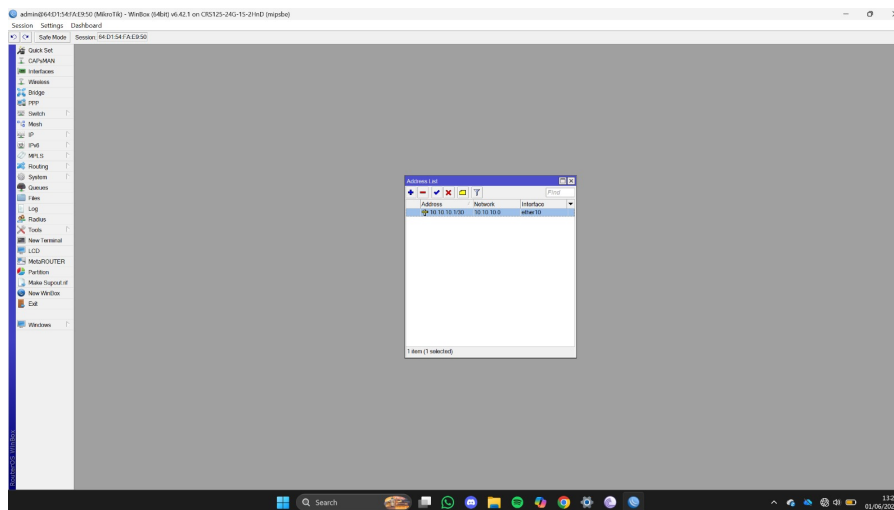
Langkah 2: Konfigurasi DHCP Client pada Router A (ether1)

Menyambungkan port ether1 Router A ke jaringan Lab untuk mendapatkan alokasi IP dinamis menuju internet.



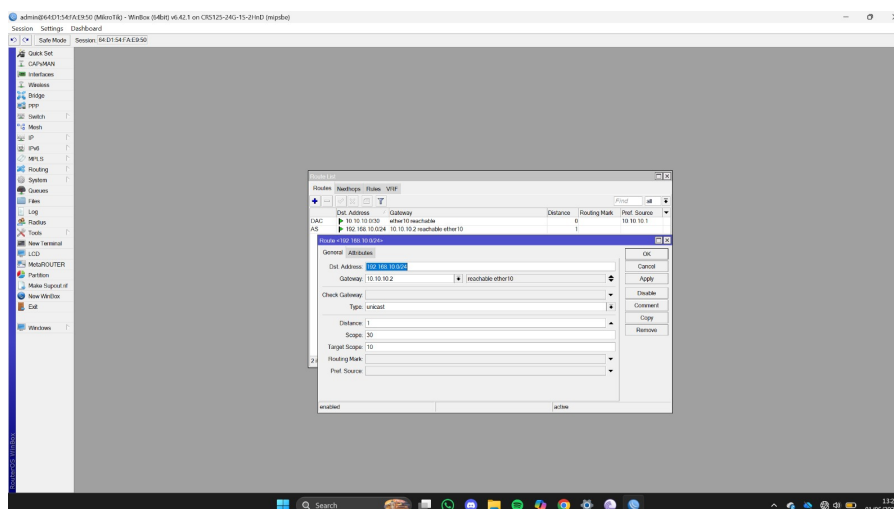
Langkah 3: Konfigurasi IP Address di Router A (ether10)

Menambahkan IP Address 10.10.10.1/30 pada antarmuka Router A (pada sesi ini menggunakan ether10 sesuai penyesuaian alat) sebagai jalur penghubung menuju Router B.



Langkah 4: Konfigurasi Routing Statis di Router A

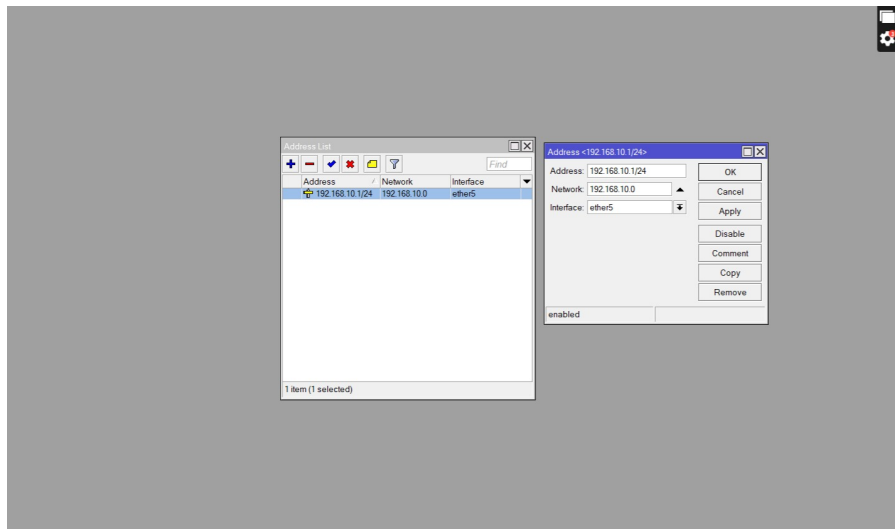
Menambahkan rute statis agar Router A mengetahui jalur menuju jaringan lokal PC Client (192.168.10.0/24) melalui gateway 10.10.10.2.



Langkah 5: Konfigurasi IP Address di Router B (ke arah Client)

Pada Router B, tambahkan IP Address yang mengarah ke PC Client. Antarmuka yang digunakan dalam sesi ini adalah ether5.

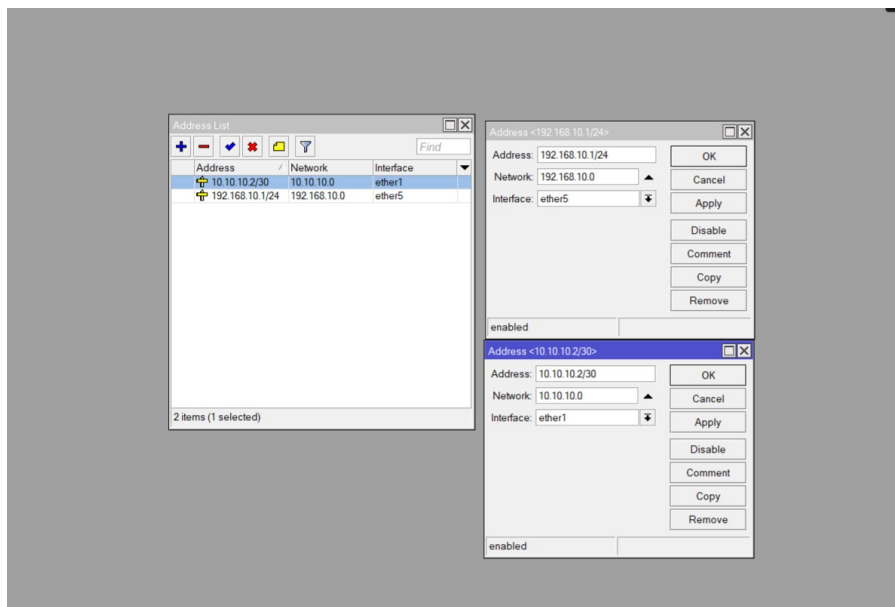
- Address: 192.168.10.1/24
- Interface: ether5



Langkah 6: Konfigurasi IP Address di Router B (ether1)

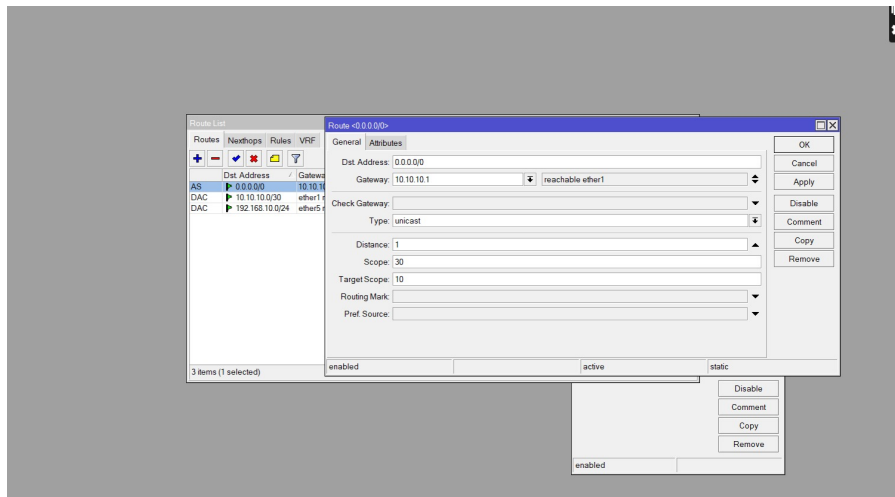
Menambahkan IP Address pada ether1 Router B yang bertindak sebagai jalur interkoneksi menuju Router A.

- Address: 10.10.10.2/30
- Interface: ether1



Langkah 7: Konfigurasi Default Route di Router B

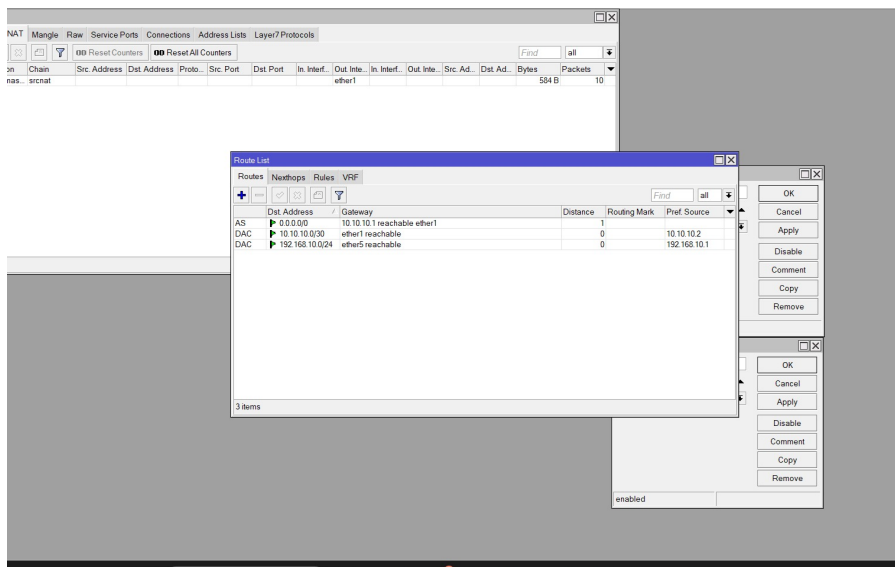
Mengatur *default route* (0.0.0.0/0) agar seluruh paket data dari jaringan lokal dilemparkan ke gerbang Router A (10.10.10.1).



Langkah 8: Konfigurasi Firewall NAT di Router B

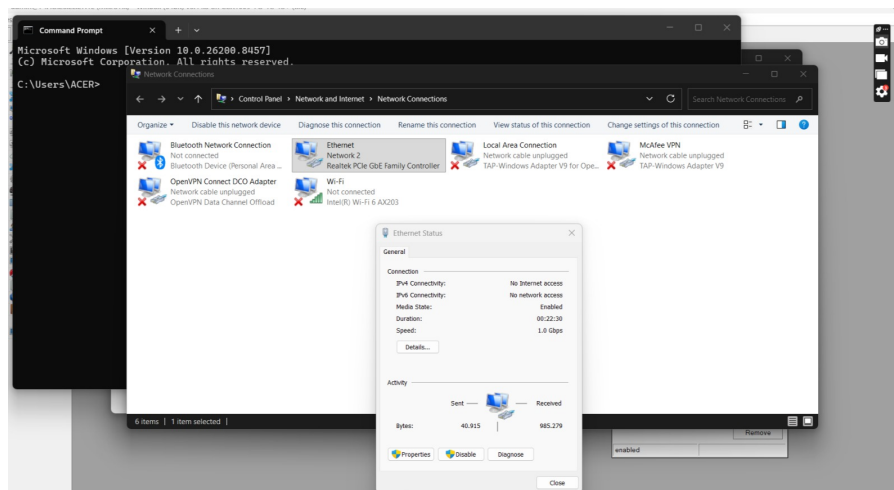
Mengaktifkan fungsionalitas NAT agar ruang alamat privat lokal dapat diterjemahkan menjadi alamat publik.

- Chain: srcnat
- Out. Interface: ether1
- Action: masquerade



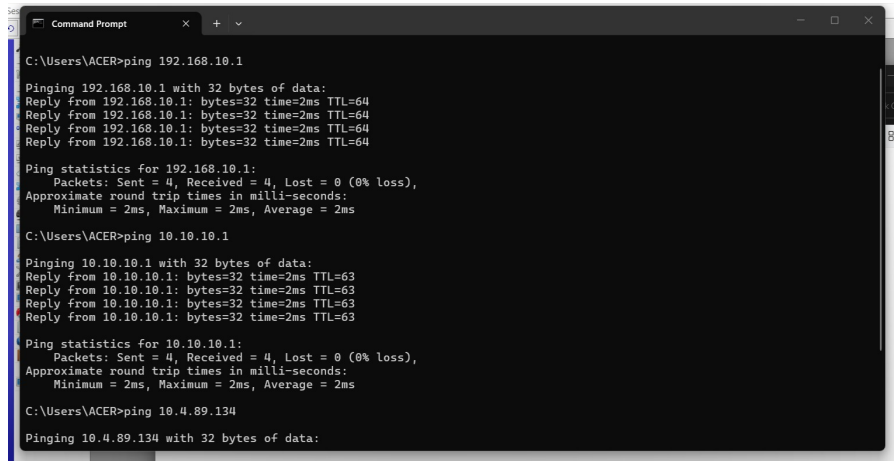
Langkah 9: Konfigurasi IP Statis di PC Client

Menyelaraskan alamat IP statis pada adapter jaringan komputer klien (Windows) agar terhubung ke gateway Router B.



Langkah 10: Pengujian Jaringan dari PC Client

Melakukan uji keterhubungan (*ping*) melalui Command Prompt menuju *gateway* lokal dan *gateway* luar. Hasilnya menunjukkan *Reply* yang menandakan konektivitas dan NAT berjalan sempurna.



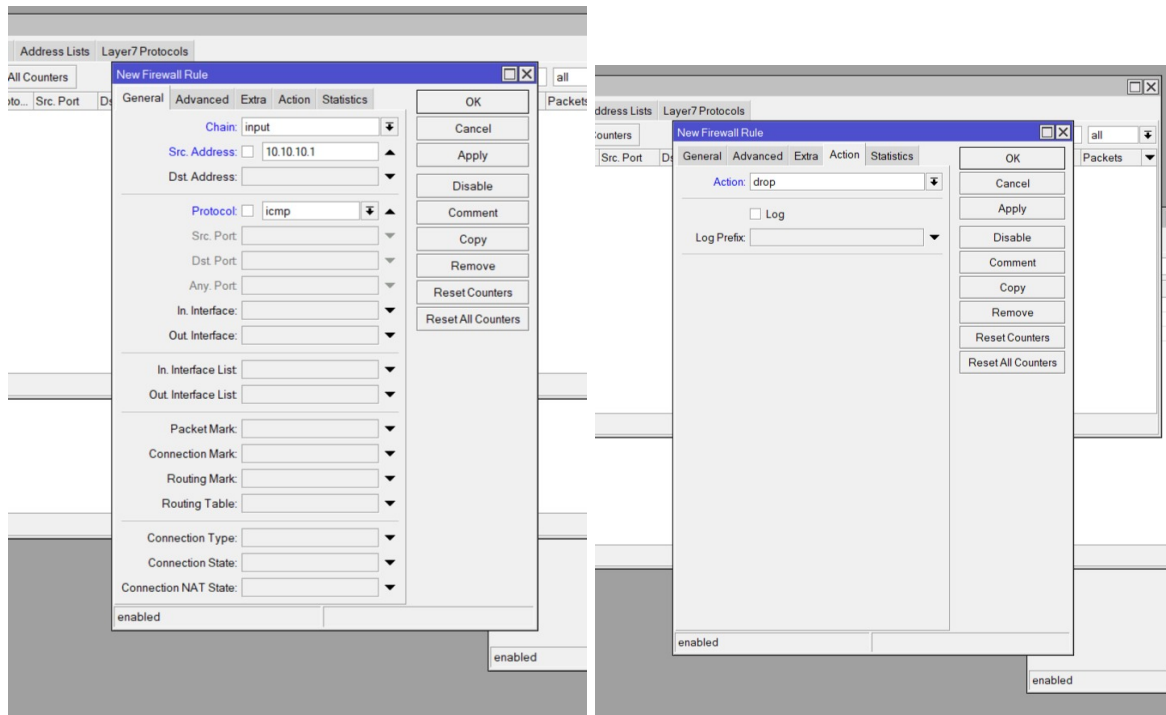
Praktikum 2: Firewall Filter (Input vs Forward)

Langkah 1: Konfigurasi Firewall Filter (Chain Input)

Membangun aturan untuk memblokir lalu lintas ICMP (*ping*) dari Router A yang ditujukan langsung ke mesin Router B.

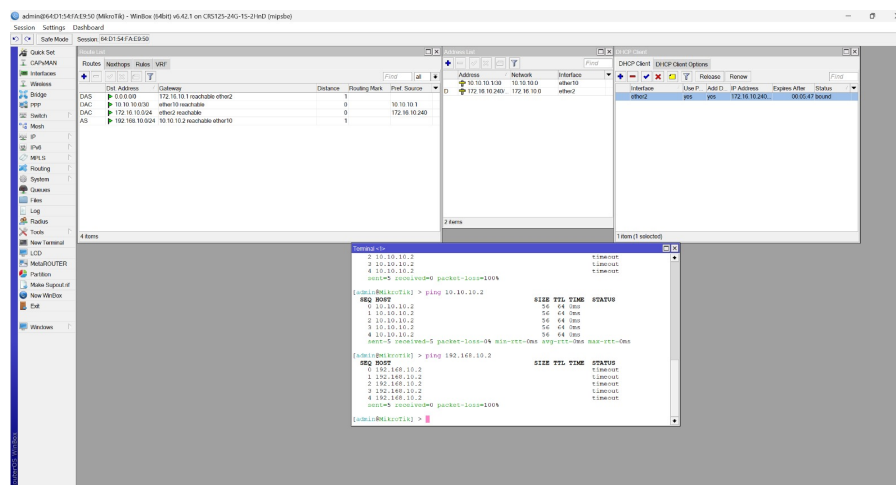
- Chain: `input`
- Src. Address: `10.10.10.1`
- Protocol: `icmp`

Aksi (*Action*) untuk aturan ini ditetapkan menjadi `drop`.



Langkah 2: Pengujian Chain Input dari Router A

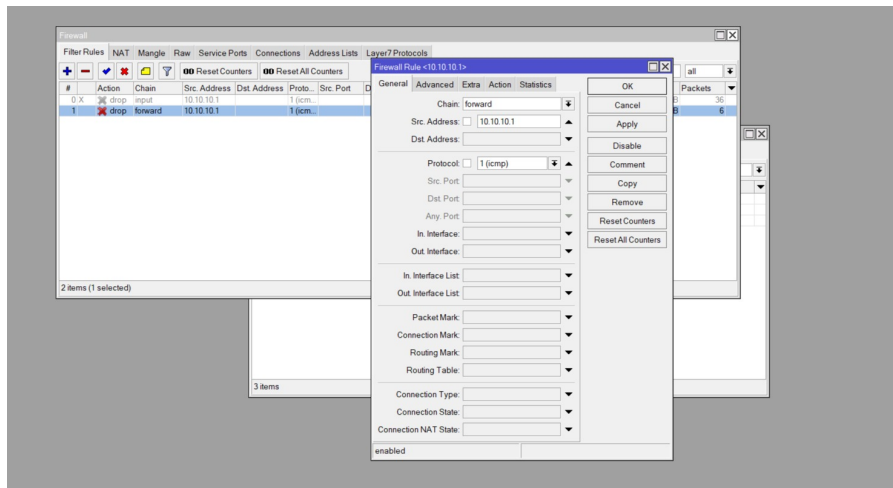
Melakukan *ping* dari Terminal Router A ke Router B. Terlihat pada terminal bahwa permintaan membuahkan hasil *Timeout*, membuktikan bahwa rantai *input* berhasil menggugurkan paket yang mengarah ke dalam dirinya sendiri.



Langkah 3: Konfigurasi Firewall Filter (Chain Forward)

Menonaktifkan aturan *input* sebelumnya, lalu membuat aturan baru guna memblokir *ping* dari Router A yang mencoba melintasi router menuju PC Client.

- Chain: forward
- Src. Address: 10.10.10.1
- Protocol: 1 (icmp)
- Action: drop



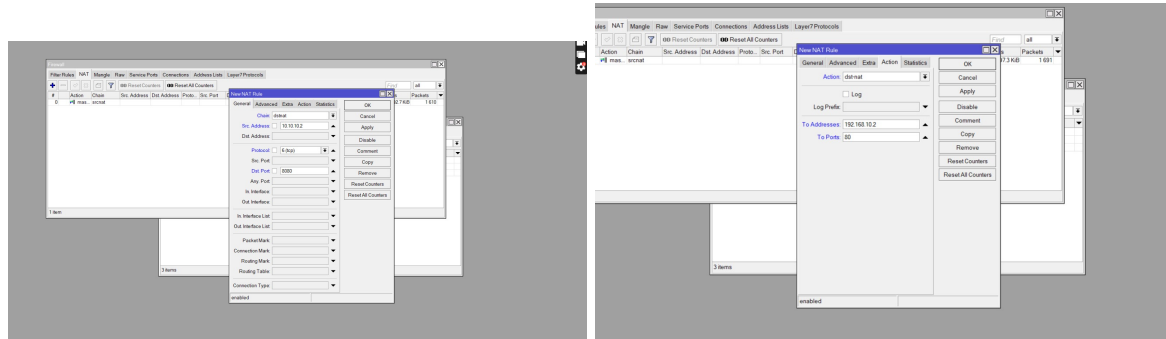
Langkah 4: Pengujian Chain Forward dari Router A

Pengujian ulang membuktikan bahwa paket ke arah Router B kembali membalas, namun paket yang mencoba menembus (numpang lewat) menuju IP 192.168.10.2 langsung dihentikan oleh dinding pelindung *forward*.

Praktikum 3: Firewall NAT Forwarding (Port Forwarding)

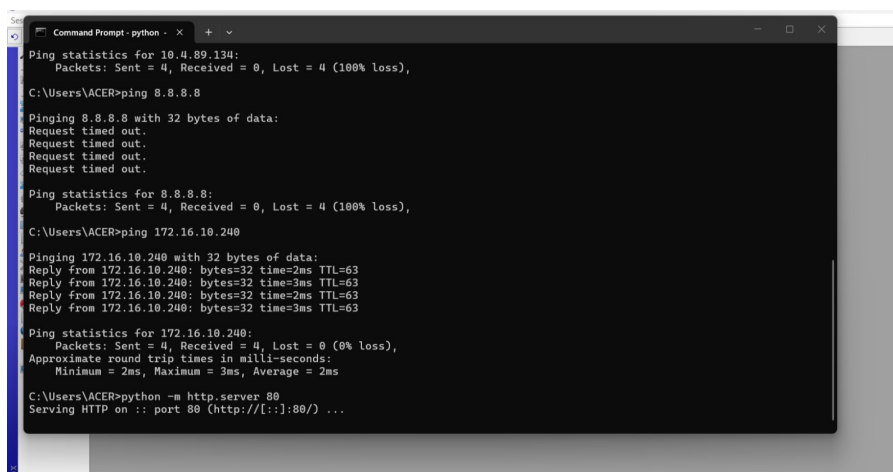
Langkah 1: Konfigurasi Port Forwarding (DstNAT) di Router B

Membelokkan permintaan lalu lintas luar yang mengarah ke IP publik Router B port 8080 menuju layanan Web Server PC Client di IP 192.168.10.2 port 80.



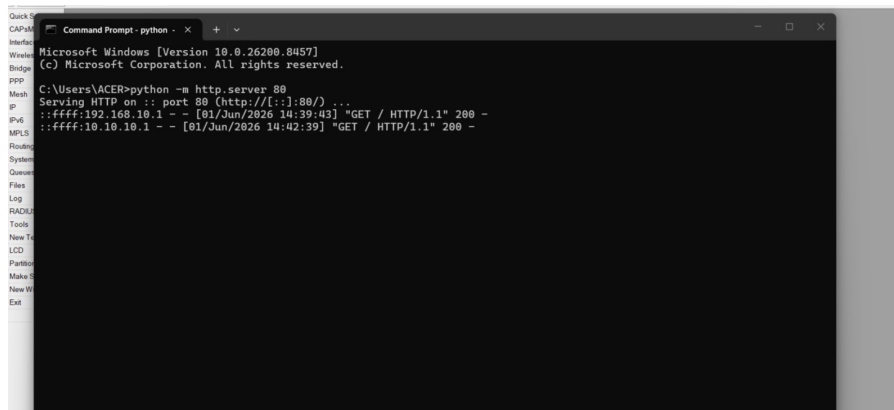
Langkah 2: Menjalankan Web Server di PC Client

Menyalakan server web instan berbasis Python di sisi terminal klien.



Langkah 3: Pengujian Port Forwarding dari Router A

Melakukan pengunduhan dari sisi luar. Pada terminal PC Client, terlihat log *request* HTTP GET masuk yang memvalidasi bahwa paket dari luar berhasil dibelokkan ke dalam.

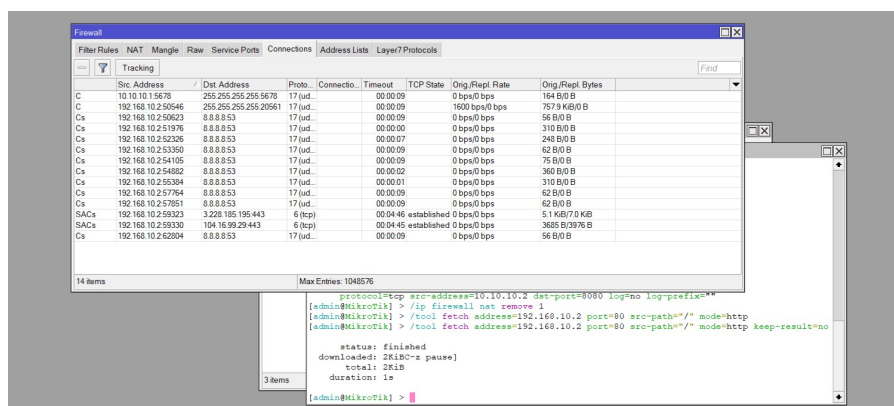


```
Microsoft Windows [Version 10.0.26290.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ACER>python -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::ffff:192.168.10.1 - - [01/Jun/2026 14:39:43] "GET / HTTP/1.1" 200 -
::ffff:192.168.10.1 - - [01/Jun/2026 14:42:39] "GET / HTTP/1.1" 200 -
```

Langkah 4: Mengamati Connection Tracking

Menganalisis tabel rekaman *Connections*. Terlihat jelas adanya translasi status sesi koneksi dari alamat luar yang dialirkan secara terarah ke mesin lokal tanpa kebocoran data.



Src Address	Dst Address	Proto	Conn	Timeout	TCP State	Orig/Repl Rate	Orig/Repl Bytes
10.10.10.1	255.255.255.255	17	ud	00:00:09	0 bps/0 bps	164 B/0 B	
192.168.10.2	255.255.255.255	17	ud	00:00:09	1600 bps/0 bps	7573 KB/0 B	
192.168.10.2	50623	8.8.8.8	17	ud	00:00:09	0 bps/0 bps	56 B/0 B
192.168.10.2	51076	8.8.8.8	17	ud	00:00:00	0 bps/0 bps	310 B/0 B
192.168.10.2	52306	8.8.8.8	17	ud	00:00:07	0 bps/0 bps	248 B/0 B
192.168.10.2	53360	8.8.8.8	17	ud	00:00:09	0 bps/0 bps	62 B/0 B
192.168.10.2	54105	8.8.8.8	17	ud	00:00:09	0 bps/0 bps	75 B/0 B
192.168.10.2	54882	8.8.8.8	17	ud	00:00:02	0 bps/0 bps	360 B/0 B
192.168.10.2	55304	8.8.8.8	17	ud	00:00:01	0 bps/0 bps	310 B/0 B
192.168.10.2	57764	8.8.8.8	17	ud	00:00:09	0 bps/0 bps	62 B/0 B
192.168.10.2	57851	8.8.8.8	17	ud	00:00:09	0 bps/0 bps	62 B/0 B
192.168.10.2	59323	3.228.185.195	6	tcp	00:04:46 established	0 bps/0 bps	51 KB/7.0 KB
192.168.10.2	59330	194.16.99.29	6	tcp	00:04:45 established	0 bps/0 bps	3665 B/3976 B
192.168.10.2	62004	8.8.8.8	17	ud	00:00:09	0 bps/0 bps	56 B/0 B

```
[admin@Mikrotik] > /ip firewall nat remove 1
[admin@Mikrotik] > /tool fetch address=192.168.10.2 port=80 src-path="/" mode=http
[admin@Mikrotik] > /tool fetch address=192.168.10.2 port=80 src-path="/" mode=http keep-result=no

status: finished
downloaded: 2KiB (-s pause)
total: 2KiB
duration: 1s
[admin@Mikrotik] >
```

2 Analisis Hasil Percobaan

Berdasarkan rentetan percobaan di atas, terbukti bahwa hierarki perlakuan lalu lintas data sangat bergantung pada penentuan *chain* yang tepat. Pada Praktikum 1, metode translasi *masquerade* berhasil menyembunyikan identitas alamat privat menjadi satu kesatuan IP publik. Penyesuaian port yang terjadi di lapangan (menggunakan ether10 dan ether5) tidak mencederai arsitektur logika jaringan sama sekali, karena NAT beroperasi di level IP (*Network Layer*).

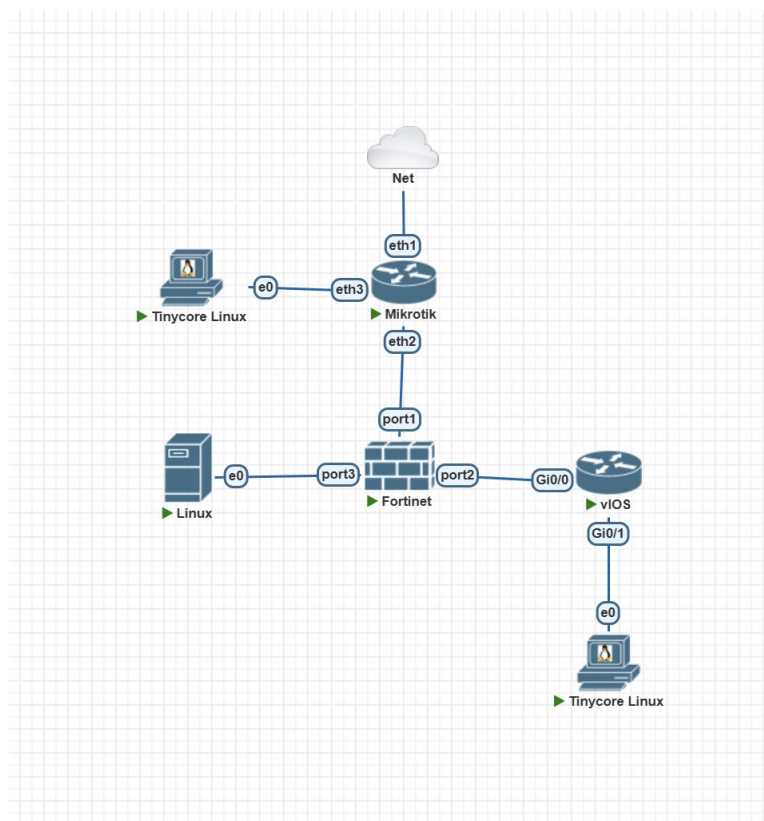
Perbedaan fundamental terungkap pada Praktikum 2: *chain input* secara eksklusif menjadi perisai bagi *host* router itu sendiri, sementara *chain forward* menjadi palang pintu bagi lalu lintas yang berupaya menyeberang antar-segmen. Pada akhirnya, NAT Forwarding (Praktikum 3) menutup kesenjangan itu dengan memberikan izin lewat secara spesifik hanya pada port 8080 yang dibelokkan menuju port 80, menjadikan layanan internal dapat diakses publik dengan risiko yang telah termitigasi.

3 Hasil Tugas Modul

Tugas modul ini mensimulasikan arsitektur jaringan *enterprise* menggunakan kolaborasi perangkat multi-vendor (FortiGate, Cisco, MikroTik) dalam topologi PNETLab.

1. Topologi Jaringan

Topologi dibagi menjadi tiga wilayah utama: WAN (Luar), LAN (Privat), dan DMZ (Peladen Publik).



2. Konfigurasi Perangkat dan Layanan (DMZ)

Konfigurasi antarmuka dan *routing* disematkan pada setiap *node*. Pada Ubuntu Server di zona DMZ, layanan Nginx dijalankan dengan injeksi teks HTML spesifik untuk kelompok.

```

Terminal
Linux * Fortinet *

Preparing to unpack .../23-libnginx-mod-mail_1.10.0-0ubuntu1.7_amd64.deb ...
Unpacking libnginx-mod-mail (1.10.0-0ubuntu1.7) ...
Selecting previously unselected package libnginx-mod-stream.
Preparing to unpack .../24-libnginx-mod-stream_1.10.0-0ubuntu1.7_amd64.deb ...
Unpacking libnginx-mod-stream (1.10.0-0ubuntu1.7) ...
Selecting previously unselected package nginx-core.
Preparing to unpack .../25-nginx-core_1.10.0-0ubuntu1.7_amd64.deb ...
Unpacking nginx-core (1.10.0-0ubuntu1.7) ...
Selecting previously unselected package nginx.
Preparing to unpack .../26-nginx_1.10.0-0ubuntu1.7_all.deb ...
Unpacking nginx (1.10.0-0ubuntu1.7) ...
Setting up libxpm6d:amd64 (1:3.5.12-1ubuntu0.20.04.2) ...
Setting up nginx-common (1.10.0-0ubuntu1.7) ...
Setting up nginx, etc:/systemd/system/nginx.service → /lib/systemd/system/nginx.service.
Setting up libjpeg62:amd64 (2:1.3.1-1ubuntu0.20.04.1) ...
Setting up libjpeg-turbo8:amd64 (2:1.3.0-2ubuntu1.7) ...
Setting up libbrotli1:amd64 (0.6.1-2ubuntu0.20.04.3) ...
Setting up fonts-dejavu-core (2.37-1) ...
Setting up libjpeg-turbo8:amd64 (2:0.3-0ubuntu1.20.04.3) ...
Setting up libjpeg9:amd64 (9c-2ubuntu0) ...
Setting up libnginx-mod-mail (1.10.0-0ubuntu1.7) ...
Setting up fontconfig-config (2.13.1-2ubuntu3) ...
Setting up libnginx-mod-stream (1.10.0-0ubuntu1.7) ...
Setting up librtfss:amd64 (4.1.0-1ubuntu17.2ubuntu0.20.04.14) ...
Setting up libfontconfig1:amd64 (2.13.1-2ubuntu3) ...
Setting up libg3:amd64 (2:2.5-3.2ubuntu2) ...
Setting up libnginx-mod-http-geoip-filter (1.10.0-0ubuntu1.7) ...
Setting up nginx-core (1.10.0-0ubuntu1.7) ...
Processing triggers for ufw (0.36-6) ...
Processing triggers for systemd (245.4-ubuntu3) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for libc-bin (2.31-0ubuntu9) ...
root@kvm:~# echo "Tunex 4 DMC Firewall_04-Samuel Hidesaki" > /var/www/html/index.html
root@kvm:~# systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /lib/systemd/system-nginx-install.
Executing: /lib/systemd/systemd-nginx-install enable nginx
root@kvm:~# systemctl restart nginx

Command 'systemctl' not found, did you mean:

command 'systemctl' from deb systemd (245.4-ubuntu3.24)
command 'systemctl' from deb systemd (1:4.3424.2)

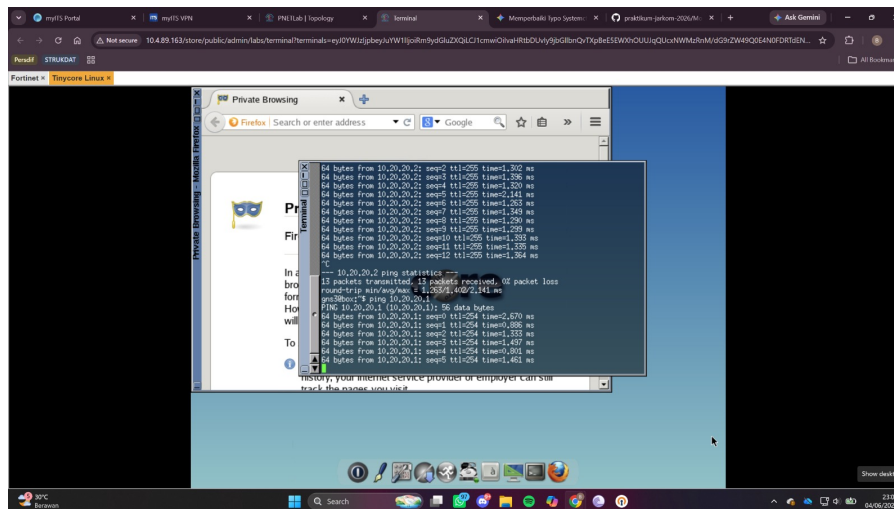
Try: apt install deb name

root@kvm:~# systemctl restart nginx
root@kvm:~#

```

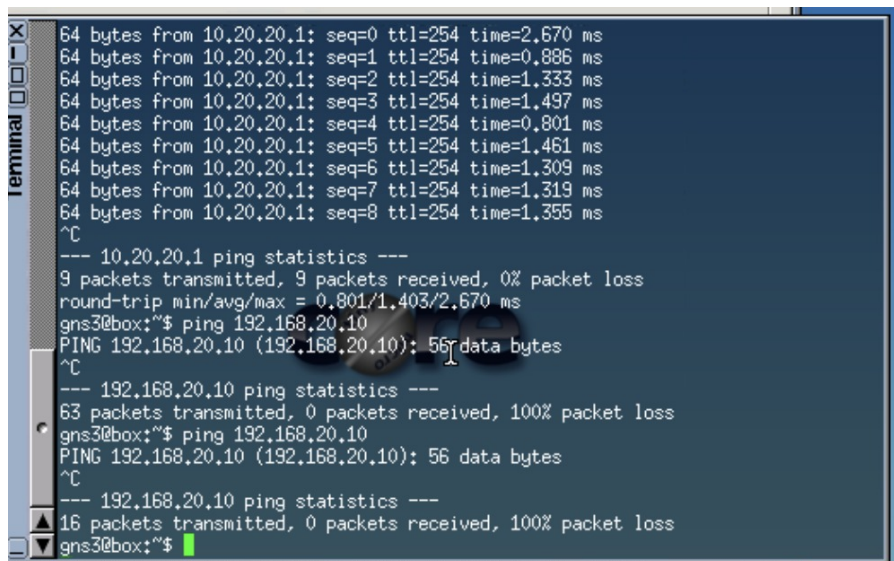
3. Pengujian Client LAN ke Gateway Cisco & FortiGate

Terkonfirmasi bahwa *host* internal (TinyCore Linux) mampu menjangkau *gateway* Cisco (10.20.20.2) dan merambat hingga kaki FortiGate (10.20.20.1) tanpa hambatan paket.



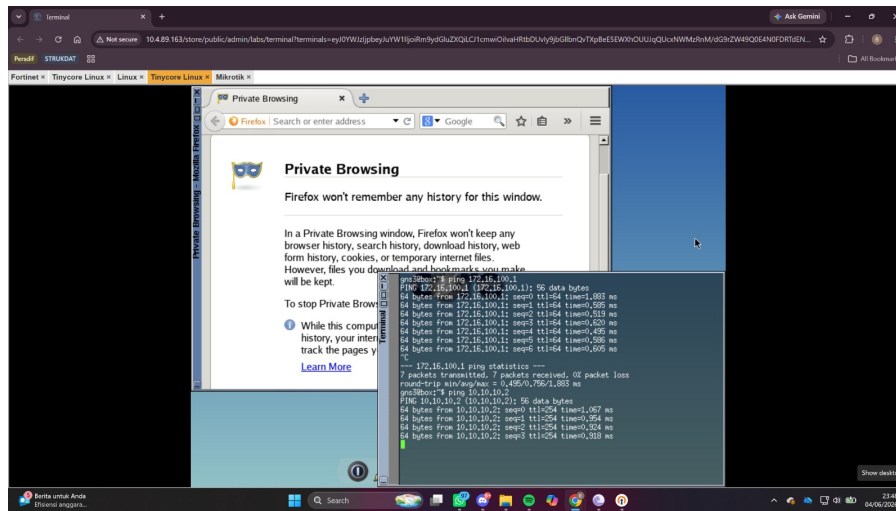
4. Pengujian Client LAN ke Zona DMZ

Paket *ping* dari LAN menembus FortiGate dan sampai ke *server* Ubuntu DMZ (192.168.20.10). Integrasi akses HTTP via penjelajah web juga mengonfirmasi *web server* berjalan merespons *request* lokal.



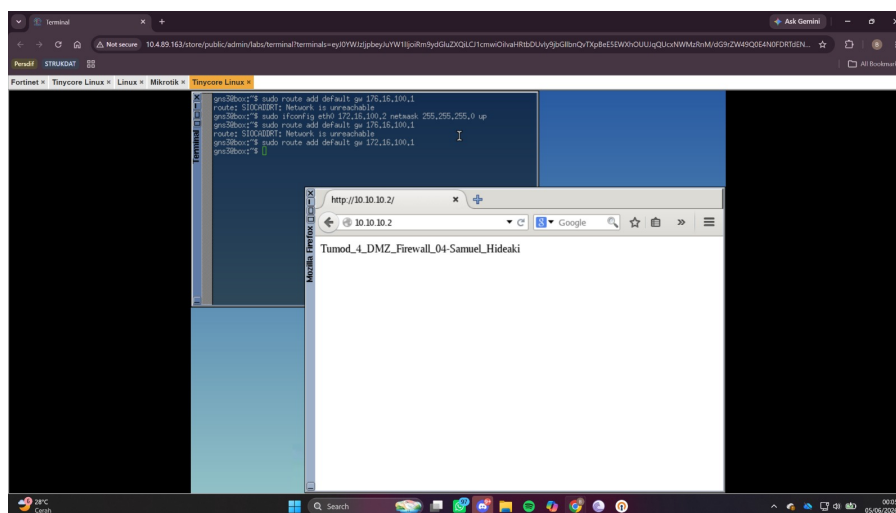
5. Pengujian Client WAN Ping ke ISP & FortiGate

Dari sisi terluar, klien berhasil menjangkau *gateway* ISP MikroTik (172.16.100.1) dan IP WAN milik FortiGate (10.10.10.2).



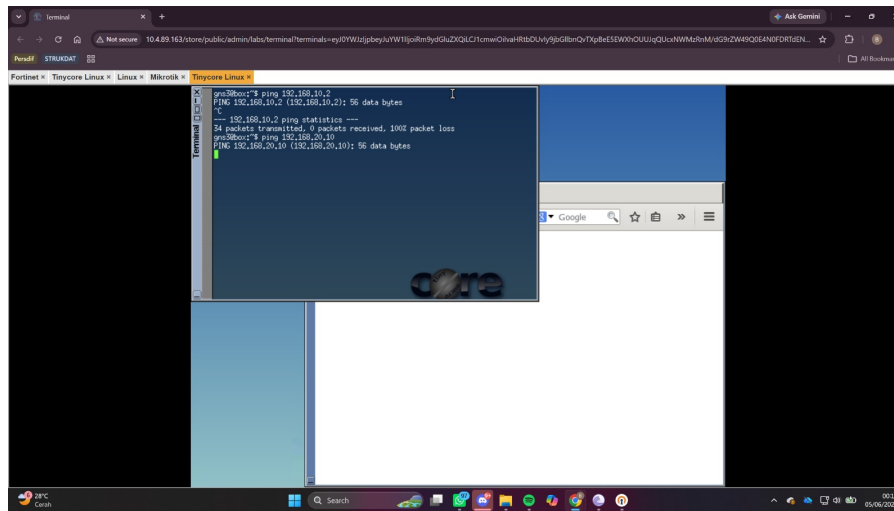
6. Pengujian Client WAN Akses HTTP (Port Forwarding DMZ)

Pembuktian utama dari *Virtual IP (VIP)* FortiGate. Saat klien WAN mengakses IP Publik 10.10.10.2, FortiGate secara senyap membelokkan *request* tersebut ke *server* DMZ, menampilkan halaman: Tumod_4_DMZ_Firewall_04-Samuel_Hideaki.



7. Pengujian Keamanan Silang (WAN ke LAN & DMZ langsung)

Sesuai rancangan keamanan sejati, upaya *ping* dari Client WAN yang secara lancang mencoba mengetuk *host* LAN (192.168.10.10) atau IP riil DMZ (192.168.20.10) sepenuhnya membentur dinding penolakan (*100% packet loss*).



4 Kesimpulan

Tembok pertahanan jaringan tidak didirikan untuk memutus komunikasi, melainkan untuk memastikan bahwa setiap percakapan data terjadi sesuai dengan aturan main yang sah. Melalui modul ini, *Connection Tracking* terbukti meningkatkan efisiensi *firewall* dengan "mengingat" status jabat tangan paket data tanpa harus memfilter ulang dari awal.

Penerapan *Destination NAT (Port Forwarding)* dan arsitektur Zona Demiliterisasi (DMZ) dengan FortiGate menegaskan prinsip isolasi; peladen dapat berdiri tegak melayani kerasnya paparan publik tanpa perlu mengorbankan integritas dan keamanan data internal di wilayah LAN. Segala akses yang tidak memiliki tiket izin secara *default* akan menemui jalan buntu.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 1: Foto Dokumentasi